☑ How do you find the spark job is struggling With **Out Of Memory Issue** ?

No error in spark logs

- Driver - stdout → No error
- Driver - stderr → No Error
- Executor - stdout → No Error
- Executor - stderr → No Error
- Memory metrics → shows normal
- Spark Ui → all looks good

But Job is failing with undefined Exceptions….

1. Executor Lost / Container Killed Messages

- **"ExecutorLostFailure"**
- **"Container killed by YARN for exceeding memory limits"**
- **"Container killed due to memory limit"**
- **"ExitCode: 137"** (Linux SIGKILL → usually memory kill)
- **"Executor exited with exit code 143/137“**

Why?

The OS or cluster manager (YARN/K8s) kills the executor due to memory crossing limits. JVM never prints OOM.

log snippet:

20/11/13 17:32:45 INFO TaskSchedulerImpl: Lost executor 5 on worker-1: ExecutorLostFailure (executor 5 exited caused by one of the running tasks)
Reason: Container killed by YARN for exceeding memory limits.
ExitCode: 137

2 Shuffle Fetch Failed Errors

- **"FetchFailedException"**
- **"Failed to connect to executor"**
- **"ShuffleBlockFetcherIterator errors"**
- **"Too much data to spill“**

This usually means:

The shuffle-producing executor died (most commonly due to memory overflow).

log snippet:

```
ERROR ShuffleBlockFetcherIterator: Exception while fetching shuffle files java.io.IOException:
FetchFailedException: Failed to fetch shuffle blocks from 10.0.0.5: Executor 7 died
   at
org.apache.spark.storage.BlockManagerMasterEndpoint.removeBlockManager(BlockManagerMaster.s
cala:...)
```

3 GC Overhead Limit Exceeded

- **"GC overhead limit exceeded"**
- **"Java heap space"**
- **"Too many GC pauses"**
- **"Full GC before allocation“**

This means the JVM spent >98% time on GC → heap is exhausted.

log snippet:
Exception in thread "main" java.lang.OutOfMemoryError: GC overhead limit exceeded
2025-11-12 10:15:32,123 WARN org.apache.spark.executor.JavaUtils: JVM started with -Xmx4g, full GC frequently triggered

4 Executor Heartbeat Timeout

- **Executor heartbeat timedout**
- **Executor has been removed**

Meaning:
The executor was stuck in GC due to memory pressure → cannot respond → Spark removes it.

This indirectly indicates memory exhaustion.

log snippet:
20/11/13 18:02:11 WARN TaskSchedulerImpl: Removing executor 3 because it has nexecutorot responded within the timeout.
20/11/13 18:02:11 INFO CoarseGrainedExecutorBackend: Driver commanded to remove 3

5 Python Worker Memory Errors

PySpark indicators:

- Python worker exited,
- Broken pipe,
- EOF from Python worker.

✔ Meaning: Python worker process killed by OS (memory).

log snippet:

```
py4j.protocol.Py4JNetworkError: Error while sending or receiving
Python worker exited unexpectedly (rc=137)
Traceback (most recent call last):
  File "/usr/lib/spark/python/pyspark/worker.py", line 62, in main
MemoryError: Unable to allocate 2000000000 bytes
```

6 Serialization Failures

Serialization issues:

- Failed to allocate memory,
- Buffer overflow.

✔ Meaning: Objects too large to serialize or insufficient memory.

log snippet:

```
ERROR SerializerInstance: Failed to serialize task 12
java.lang.OutOfMemoryError: Java heap space
        at java.io.ObjectOutputStream$HandleTable.assign(ObjectOutputStream.java:...)
Caused by: java.io.IOException: Failed to allocate memory while serializing large object
```

7 Spill Warnings

Spill warnings:

- Spilling to disk,
- Sort buffer full.

✔ Meaning: Continuous spills are precursor to OOM.

log snippet:
WARN ShuffleExternalSorter: Spilling map output to disk (will be unsorted), current buffer size reached limit
20/11/13 12:45:00 WARN UnsafeExternalSorter: Memory usage is high, spilling to disk. Total spills: 3

8 Kubernetes / YARN Memory Kill

YARN:

- **Exit Code 137** → memory kill
- **Exit Code 143** → killed by YARN
- **"Container [pid] was terminated due to exceeding physical memory”**

Kubernetes:

- **"OOMKilled"** (in pod events)
- **"Killed: memory cgroup“**

No Spark logs show OOM, but pod-level logs confirm it.

log snippet:

```
Kubernetes event: Pod spark-executor-6 - OOMKilled
Container spark-executor terminated with exit code 137 (memory limit exceeded)
YARN: Container [pid=12345] was terminated due to exceeding physical memory limits
```